IMPLEMENTASI JOB QUEUE DAN MESSAGE BROKERING MENGGUNAKAN CELERY DAN RABBITMQ UNTUK ANALISIS LOG SIEM PADA CLUSTER COMPUTING

Laporan Tugas 2



Oleh
Afif faisal alamsyah
26051905018
Oky Dwi Prasetyo
26051905034

KOMPUTASI BERBASIS JARINGAN
UNIVERSITAS NEGERI SURABAYA
FAKULTAS TEKNIK
PROGRAM STUDI S2 INFORMATIKA
2026

1. Pendahuluan

Dalam arsitektur Security Operations Center (SOC) modern, pemrosesan log keamanan yang masif memerlukan sistem antrean (job queue) yang andal. Pada tugas sebelumnya, pendekatan load balancing dibangun secara manual menggunakan Socket Programming. Meskipun fungsional, pendekatan tersebut memiliki keterbatasan dalam skalabilitas, toleransi kesalahan (fault tolerance), dan manajemen routing tugas yang kompleks.

Laporan ini mendokumentasikan implementasi arsitektur Broker-Worker menggunakan protokol AMQP melalui RabbitMQ sebagai Message Broker dan Celery sebagai antrean tugas terdistribusi (distributed task queue). Implementasi ini menyempurnakan sistem distribusi beban untuk dua jenis tugas SIEM: evaluasi ketersediaan agen (SHORT task) dan prediksi model Machine Learning untuk Alert Triage (LONG task), serta menambahkan antarmuka pemantauan (monitoring) secara real-time menggunakan Flower.

2. Arsitektur dan Desain Sistem

2.1 Topologi Sistem Celery-RabbitMQ

Arsitektur jaringan bergeser dari komunikasi socket langsung menjadi arsitektur berbasis antrean pesan terpusat:

1. **Producer (Agent SIEM) / producer.py:** Skrip klien yang membangkitkan data masukan (log) dan mengirimkannya ke RabbitMQ secara asinkron (delay).
2. **Message Broker (RabbitMQ):** Menerima pesan dari producer dan menyimpannya dalam antrean (queue) menggunakan skema task routing yang dikonfigurasi.
3. **Celery Workers / task.py:** Node pekerja yang mengonsumsi (consume) pesan dari antrean RabbitMQ spesifik (q_short_computation atau q_long_computation) dan mengeksekusi beban kerja.
4. **Dashboard Monitor (Flower):** Perangkat lunak pemantauan berbasis web yang terhubung langsung ke RabbitMQ untuk memvisualisasikan status antrean (queue), performa worker, dan riwayat tugas secara real-time.

2.2 Konsep Penanganan Pesan (Message Handling)

Sistem mengadopsi protokol AMQP (Advanced Message Queuing Protocol) yang mendukung beberapa skema perutean (exchange):

- **Fan-Out (Broadcast):** Mengirimkan salinan pesan ke semua antrean yang terhubung tanpa mempedulikan routing key. Diimplementasikan di SOC untuk menyebarkan pembaruan konfigurasi global secara serentak.
- **Topic: Topic & Wildcard:** Merutekan pesan berdasarkan kecocokan pola pada *routing key* (misalnya `siem.log.#`). Pada tugas ini, mekanisme perutean disederhanakan dengan mendefinisikan kamus `task_routes` secara eksplisit di tingkat konfigurasi Celery agar tugas terarah secara otomatis ke antrean masing-masing.

3. Implementasi Kode Program

3.1 celeryconfig.py (Konfigurasi Broker & Routing)

```
broker_url = 'amqp://guest:guest@localhost:5672//'

result_backend = 'rpc://'



# Task Routing: Spesifikasi destinasi antrean (queue)

task_routes = {

   'tasks.process_ml_triage': {'queue': 'q_long_computation'},

   'tasks.process_wazuh_ping': {'queue': 'q_short_computation'},

}
```

3.2 tasks.py (Celery Worker)

```
from celery import Celery
import time
import json
import celeryconfig

# Inisialisasi aplikasi Celery dan memuat konfigurasi
app = Celery("siem_celery")
app.config_from_object('celeryconfig')

@app.task(bind=True)
def process_ml_triage(self, payload):
   """
   Task LONG: Analisis Machine Learning untuk Alert Triage
   Akan dirutekan otomatis ke queue 'q_long_computation'
   """
   time.sleep(3) # Simulasi komputasi model ML
   try:
      data = json.loads(payload)
      risk_score = 95 if data.get("attack_type") in ["SQLi", "DoS"] else 30
      return f"TRIAGE OK: Agent {data.get('agent_name')} | Risk: {risk_score}%"
   except Exception as e:
      return f"TRIAGE FAIL: {e}"

@app.task(bind=True)
def process_wazuh_ping(self, payload):
   """
   Task SHORT: Pengecekan status ketersediaan agen (Health-check)
   Akan dirutekan otomatis ke queue 'q_short_computation'
```

```
    """
    try:
        data = json.loads(payload)
        return f"PING OK: Agent {data.get('agent_id')} Active"
    except Exception:
        return "PING OK: Agent Active"
```

3.3 producer.py (Input Generator)

```
import json
import time
from tasks import process_ml_triage, process_wazuh_ping

def generate_siem_traffic():
    print("=== SIEM Log Generator (Producer) Memulai Transmisi ===")

    # Simulasi 5 antrean log beruntun
    log_events = [
        {"type": "ping", "data": {"agent_id": "WAF-Node-01", "status": "active"}},
        {"type": "triage", "data": {"agent_name": "Web-App", "attack_type": "SQLi"}},
        {"type": "ping", "data": {"agent_id": "WAF-Node-02", "status": "active"}},
        {"type": "triage", "data": {"agent_name": "DB-Core", "attack_type": "DoS"}},
        {"type": "ping", "data": {"agent_id": "WAF-Node-03", "status": "active"}}
    ]

    async_results = []

    for event in log_events:
        payload_str = json.dumps(event["data"])

        # Klien membuat task yang dikirim ke RabbitMQ secara asynchronous
        if event["type"] == "ping":
            print(f"[Producer] Mengirim SHORT Task (Ping) -> {payload_str}")
            # Pemanggilan delay() melempar pesan ke antrean broker
            res = process_wazuh_ping.delay(payload_str)
        else:
            print(f"[Producer] Mengirim LONG Task (ML Triage) -> {payload_str}")
            res = process_ml_triage.delay(payload_str)

        async_results.append(res)
        time.sleep(0.5)

    print("\n[Producer] Seluruh pesan telah dikirim ke Broker.")
    print("[Producer] Menunggu hasil dari Worker...\n")

    # Menarik hasil dari result backend
    for res in async_results:
        # get() akan memblokir hingga worker selesai memproses pesan terkait
        final_output = res.get()
        print(f"Hasil Eksekusi Worker: {final_output}")
```

```
if __name__ == "__main__":
    generate_siem_traffic()
```

4. Pengujian dan Analisis Hasil

4.1 Lingkungan Pengujian (*Testbed Environment*)

Pengujian fungsional dan performa arsitektur *Job Queue* ini dilakukan pada lingkungan lokal (*localhost*) dengan rincian spesifikasi:

- **Sistem:** OS Windows & Docker
- **Virtual Environment:** Python 3.10 dengan manajemen paket `uv` dan `venv`
- **Message Broker:** RabbitMQ 3 Management (dijalankan via Docker Desktop pada port `5672` dan `15672`)
- **Monitoring Tool:** Flower v2.0.1 (diakses melalui port `5555`)
- **Catatan Kompatibilitas OS:** Untuk mengatasi kendala proses *multiprocessing fork/billiard* bawaan Celery pada sistem Windows yang memunculkan *error* `PermissionError: [WinError 5] Access is denied`, eksekusi Celery worker dikonfigurasi menggunakan parameter `--pool=solo`.

4.2 Perintah Eksekusi Lengkap (*Step-by-Step Commands*)

Eksekusi sistem dilakukan secara konkuren menggunakan 4 jendela terminal terpisah dengan langkah-langkah berikut:

- **Langkah 1: Menyalakan Message Broker (Docker RabbitMQ)**

  ```
  docker run -d --name rabbitmq-server -p 5672:5672 -p 15672:15672 rabbitmq:3-management
  ```

- **Langkah 2: Mengaktifkan Virtual Environment & Install Dependensi**

  ```
  Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process
  .\myvenv\Scripts\Activate.ps1
  pip install -r requirements.txt
  ```

  Langkah 3: Menjalankan Worker Komputasi Ringan (Terminal 1)

  ```
  celery -A tasks worker -Q q_short_computation --loglevel=INFO --pool=solo -n worker_short@%h
  ```

```
 -------------- worker_short@7000000NMRS51 v5.5.2 (immunity)
--- ***** -----
-- ******* ---- Windows-11-10.0.22631-SP0 2026-11-09 08:51:12
- *** --- * ---
- ** ---------- [config]
- ** ---------- .> app:         siem_celery:0x2c10b647770
- ** ---------- .> transport:   amqp://guest:**@localhost:5672//
- ** ---------- .> results:     rpc://
- *** --- * --- .> concurrency: 8 (solo)
-- ******* ---- .> task events: OFF (enable -E to monitor tasks in this worker)
--- ***** -----
 -------------- [queues]
                .> q_short_computation exchange=q_short_computation(direct) key=q_short_computation


[tasks]
  . tasks.process_ml_triage
  . tasks.process_wazuh_ping
```

Gambar 4.1 *Log aktif worker_short pada antrean q_short_computation*

- **Langkah 4: Menjalankan Worker Komputasi Berat (Terminal 2)**

```
celery -A tasks worker -Q q_long_computation --loglevel=INFO --
pool=solo -n worker_ml@%h
```

```
 -------------- worker_ml@7000000NMRS51 v5.5.2 (immunity)
--- ***** -----
-- ******* ---- Windows-11-10.0.22631-SP0 2026-11-09 08:51:24
- *** --- * ---
- ** ---------- [config]
- ** ---------- .> app:         siem_celery:0x24981997770
- ** ---------- .> transport:   amqp://guest:**@localhost:5672//
- ** ---------- .> results:     rpc://
- *** --- * --- .> concurrency: 8 (solo)
-- ******* ---- .> task events: OFF (enable -E to monitor tasks in this worker)
--- ***** -----
 -------------- [queues]
                .> q_long_computation exchange=q_long_computation(direct) key=q_long_computation


[tasks]
  . tasks.process_ml_triage
  . tasks.process_wazuh_ping
```

Gambar 4.2 *Log aktif worker_ml pada antrean q_long_computation*

- **Langkah 5: Menjalankan Dashboard Flower (Terminal 3)**

```
http://localhost:5555/
```



Gambar 4.3 *Antarmuka monitoring Flower menampilkan status worker dan task history*

- **Langkah 6: Menjalankan Producer / Generator Log (Terminal 4)**

  ```
  python producer.py
  ```

```
(myvenv) PS C:\Tugas2_KBJ_Celery> python producer.py
=== SIEM Log Generator (Producer) Memulai Transmisi ===
[Producer] Mengirim SHORT Task (Ping) -> {"agent_id": "WAF-Node-01", "status": "active"}
[Producer] Mengirim LONG Task (ML Triage) -> {"agent_name": "Web-App", "attack_type": "SQLi"}
[Producer] Mengirim SHORT Task (Ping) -> {"agent_id": "WAF-Node-02", "status": "active"}
[Producer] Mengirim LONG Task (ML Triage) -> {"agent_name": "DB-Core", "attack_type": "DoS"}
[Producer] Mengirim SHORT Task (Ping) -> {"agent_id": "WAF-Node-03", "status": "active"}

[Producer] Seluruh pesan telah dikirim ke Broker.
[Producer] Menunggu hasil dari Worker...

Hasil Eksekusi Worker: PING OK: Agent WAF-Node-01 Active
Hasil Eksekusi Worker: TRIAGE OK: Agent Web-App | Risk: 95%
Hasil Eksekusi Worker: PING OK: Agent WAF-Node-02 Active
Hasil Eksekusi Worker: TRIAGE OK: Agent DB-Core | Risk: 95%
Hasil Eksekusi Worker: PING OK: Agent WAF-Node-03 Active
```

Gambar 4.4 *Eksekusi producer.py dan penerimaan hasil RPC backend*

4.3 Analisis Hasil dan Performa Sistem

1. **Pemisahan Antrean Berbasis Routing:** Saat `producer.py` mengeksekusi pengiriman, pesan langsung dipilah secara otomatis oleh *broker* RabbitMQ. Tugas `WAZUH_PING` masuk ke antrean `q_short_computation` dan diselesaikan seketika (0.001 detik). Sebaliknya, tugas `ML_ALERT_TRIAGE` masuk ke antrean `q_long_computation` dan diproses dengan latensi penundaan simulasi komputasi selama 3 detik.
2. **Eliminasi Masalah Blocking:** Berbeda dengan pemrosesan *socket* sekuensial atau *single-threaded*, penggunaan *Job Queue* berbasis Celery memastikan bahwa penundaan pada tugas ML tidak mengganggu kelancaran lalu lintas pengecekan kesehatan agen keamanan lainnya.
3. **Observabilitas via Flower:** Dasbor Flower memberikan visibilitas penuh secara *real-time*, mencatat UUID unik setiap tugas, durasi eksekusi aktual (*runtime*), status akhir bernilai `SUCCESS`, serta identitas node *worker* yang bertanggung jawab menyelesaikan transaksi tersebut.

5. KESIMPULAN DAN SARAN

5.1 Kesimpulan

1. Implementasi *Job Queue* menggunakan Celery dan RabbitMQ terbukti jauh lebih tangguh, terstruktur, dan skalabel dibandingkan pendekatan *socket programming* manual.
2. Mekanisme *Task Routing* berhasil mengisolasi beban kerja berat dan ringan secara sempurna pada *cluster worker* yang berbeda.
3. Integrasi perangkat pemantauan Flower memberikan visibilitas operasional yang sangat baik untuk melacak performa sistem dan riwayat eksekusi tugas.

5.2 Saran

1. Untuk implementasi pada lingkungan produksi (*production*), disarankan menggunakan sistem operasi berbasis Linux agar pemanfaatan fitur *multiprocessing* (`prefork` pool) bawaan Celery dapat berjalan optimal dengan tingkat konkurensi tinggi.
2. Menerapkan enkripsi SSL/TLS serta manajemen hak akses kredensial yang aman pada koneksi RabbitMQ dan Flower guna melindungi integritas data log di jaringan SOC.